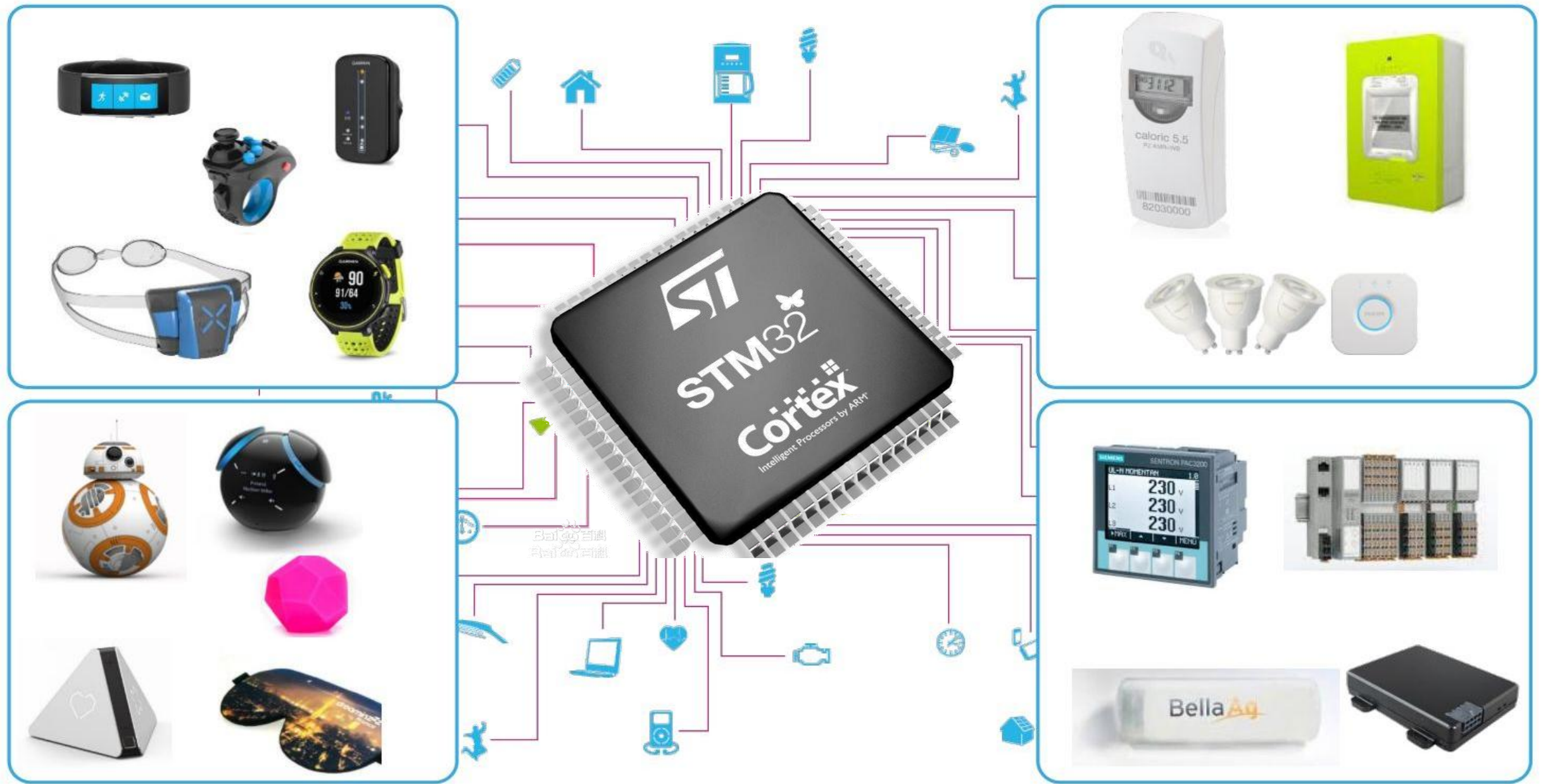
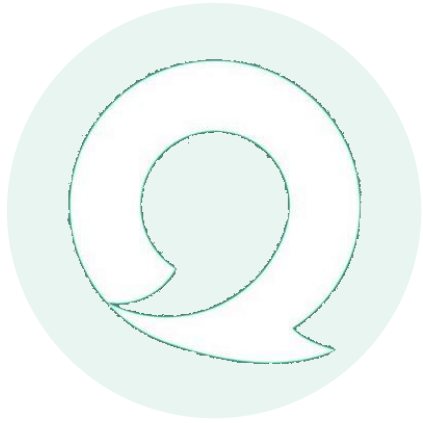




Instructor : WangXiaojing

Email : wangxj@sustech.edu.cn



Lecture4

Program based on FWLib

Topics

- Depth Analysis of FWLib
- Further Improving of--_
LED Program based on FWLib
- Key Detection--->Control LED on/off

Part I : Depth Analysis of FWLib

main.c

```
#include "stm32f10x.h" //
int main(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    // 打开 GPIOB 端口的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB,ENABLE);
    // 配置IO口为输出
    GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    // 控制 ODR 寄存器
    GPIO_ResetBits(GPIOB,GPIO_Pin_5); //LED on
    //GPIO_SetBits(GPIOB,GPIO_Pin_5); //LED off
}
```



What's RCC ?

How did I know there's such a function ?

Where should we find this function?

How to configure the parameters ?

 misc.c	 misc.h
 stm32f10x_adc.c	 stm32f10x_adc.h
 stm32f10x_bkp.c	 stm32f10x_bkp.h
 stm32f10x_can.c	 stm32f10x_can.h
 stm32f10x_cec.c	 stm32f10x_cec.h
 stm32f10x_crc.c	 stm32f10x_crc.h
 stm32f10x_dac.c	 stm32f10x_dac.h
 stm32f10x_dbgmcu.c	 stm32f10x_dbgmcu.h
 stm32f10x_dma.c	 stm32f10x_dma.h
 stm32f10x_exti.c	 stm32f10x_exti.h
 stm32f10x_flash.c	 stm32f10x_flash.h
 stm32f10x_fsmc.c	 stm32f10x_fsmc.h
 stm32f10x_gpio.c	 stm32f10x_gpio.h
 stm32f10x_i2c.c	 stm32f10x_i2c.h
 stm32f10x_iwdg.c	 stm32f10x_iwdg.h
 stm32f10x_pwr.c	 stm32f10x_pwr.h
 stm32f10x_rcc.c	 stm32f10x_rcc.h
 stm32f10x_rtc.c	 stm32f10x_rtc.h
 stm32f10x_sdio.c	 stm32f10x_sdio.h
 stm32f10x_spi.c	 stm32f10x_spi.h
 stm32f10x_tim.c	 stm32f10x_tim.h
 stm32f10x_usart.c	 stm32f10x_usart.h
 stm32f10x_wwdg.c	 stm32f10x_wwdg.h

.c 文件：函数的具体实现；

.h 文件：头文件，函数声明，define语句以及结构体定义；

 7.3 RCC registers	
 7.3.1 Clock control register (RCC_CR)	
 7.3.2 Clock configuration register (RCC_CFGR)	
 7.3.3 Clock interrupt register (RCC_CIR)	
 7.3.4 APB2 peripheral reset register (RCC_APB2RSTR)	
 7.3.5 APB1 peripheral reset register (RCC_APB1RSTR)	
 7.3.6 AHB peripheral clock enable register (RCC_AHBENR)	
 7.3.7 APB2 peripheral clock enable register (RCC_APB2ENR)	
 7.3.8 APB1 peripheral clock enable register (RCC_APB1ENR)	
 7.3.9 Backup domain control register (RCC_BDCR)	
 7.3.10 Control/status register (RCC_CSR)	



```
676 #else
677 void RCC_OTGFSCLKConfig(uint32_t RCC_OTGFSCLKSource);
678 #endif /* STM32F10X_CL */
679
680 void RCC_ADCCLKConfig(uint32_t RCC_PCLK2);
681
682 #ifdef STM32F10X_CL
683 void RCC_I2S2CLKConfig(uint32_t RCC_I2S2CLKSource);
684 void RCC_I2S3CLKConfig(uint32_t RCC_I2S3CLKSource);
685 #endif /* STM32F10X_CL */
686
687 void RCC_LSEConfig(uint8_t RCC_LSE);
688 void RCC_LSICmd(FunctionalState NewState);
689 void RCC_RTCCLKConfig(uint32_t RCC_RTCCLKSource);
690 void RCC_RTCCLKCmd(FunctionalState NewState);
691 void RCC_GetClocksFreq(RCC_ClocksTypeDef* RCC_Clocks);
692 void RCC_AHBPeriphClockCmd(uint32_t RCC_AHBPeriph, FunctionalState NewState);
693 void RCC_APB2PeriphClockCmd(uint32_t RCC_APB2Periph, FunctionalState NewState);
694 void RCC_APB1PeriphClockCmd(uint32_t RCC_APB1Periph, FunctionalState NewState);
695
696 #ifdef STM32F10X_CL
697 void RCC_AHBPeriphResetCmd(uint32_t RCC_AHBPeriph, FunctionalState NewState);
698 #endif /* STM32F10X_CL */
699
700 void RCC_APB2PeriphResetCmd(uint32_t RCC_APB2Periph, FunctionalState NewState);
701 void RCC_APB1PeriphResetCmd(uint32_t RCC_APB1Periph, FunctionalState NewState);
702 void RCC_BackupResetCmd(FunctionalState NewState);
703 void RCC_ClockSecuritySystemCmd(FunctionalState NewState);
704 void RCC_MCOConfig(uint8_t RCC_MCO);
705 FlagStatus RCC_GetFlagStatus(uint8_t RCC_FLAG);
706 void RCC_ClearFlag(void);
707 ITStatus RCC_GetITStatus(uint8_t RCC_IT);
708 void RCC_ClearITPendingBit(uint8_t RCC_IT);
709
```

Split Window horizontally

Toggle Header/Code File

Go To Definition Of 'RCC_APB2PeriphClockCmd' F12

Go To Next Reference To 'RCC_APB2PeriphClockCmd' Ctrl+Num +

Go To Previous Reference To 'RCC_APB2PeriphClockCmd' Ctrl+Num -

Insert/Remove Bookmark Ctrl+F2

Undo Ctrl+Z

Redo Ctrl+Y

Cut Ctrl+X

Copy Ctrl+C

Paste Ctrl+V

Select All Ctrl+A

Outlining

Advanced

```
stm32f10x_rcc.c
1078 }
1079
1080 /**
1081  * @brief Enables or disables the High Speed APB (APB2) peripheral clock.
1082  * @param RCC_APB2Periph: specifies the APB2 peripheral to gates its clock.
1083  * This parameter can be any combination of the following values:
1084  * @arg RCC_APB2Periph_AFIO, RCC_APB2Periph_GPIOA, RCC_APB2Periph_GPIOB,
1085  * RCC_APB2Periph_GPIOC, RCC_APB2Periph_GPIOD, RCC_APB2Periph_GPIOE,
1086  * RCC_APB2Periph_GPIOF, RCC_APB2Periph_GPIOG, RCC_APB2Periph_ADC1,
1087  * RCC_APB2Periph_ADC2, RCC_APB2Periph_TIM1, RCC_APB2Periph_SPI1,
1088  * RCC_APB2Periph_TIM8, RCC_APB2Periph_USART1, RCC_APB2Periph_ADC3,
1089  * RCC_APB2Periph_TIM15, RCC_APB2Periph_TIM16, RCC_APB2Periph_TIM17,
1090  * RCC_APB2Periph_TIM9, RCC_APB2Periph_TIM10, RCC_APB2Periph_TIM11
1091  * @param NewState: new state of the specified peripheral clock.
1092  * This parameter can be: ENABLE or DISABLE.
1093  * @retval None
1094  */
1095 void RCC_APB2PeriphClockCmd(uint32_t RCC_APB2Periph, FunctionalState NewState)
1096 {
1097     /* Check the parameters */
1098     assert_param(IS_RCC_APB2_PERIPH(RCC_APB2Periph));
1099     assert_param(IS_FUNCTIONAL_STATE(NewState));
1100     if (NewState != DISABLE)
1101     {
1102         RCC->APB2ENR |= RCC_APB2Periph;
1103     }
1104     else
1105     {
1106         RCC->APB2ENR &= ~RCC_APB2Periph;
1107     }
1108 }
1109
```

断言函数

RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, FunctionalState NewState)

RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2Periph_GPIOB, FunctionalState NewState)

main.c

```
#include "stm32f10x.h" //
int main(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    // 打开 GPIOB 端口的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB,ENABLE);
    // 配置IO口为输出
    GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    // 控制 ODR 寄存器
    GPIO_ResetBits(GPIOB,GPIO_Pin_5); //LED on
    //GPIO_SetBits(GPIOB,GPIO_Pin_5); //LED off
}
```

What's *GPIO_init* ?



How did I know there's such a function ?

Where should we find this function?

How to configure the parameters ?

stm32f10x_gpio.c

断言函数

```
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

如果函数参数填写的是**结构体指针类型**，先定义该结构体变量，在通过这个结构体变量给成员赋值，再将这个结构体变量的**地址**填入函数！

给结构体成员赋值时，可利用断言函数和结构体定义的成员类型（枚举类型）

```
#include "stm32f10x.h" //
int main(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    // 打开 GPIOB 端口的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB,ENABLE);
    // 配置IO口为输出
    GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    // 控制 ODR 寄存器
    GPIO_ResetBits(GPIOB,GPIO_Pin_5); //LED on
    //GPIO_SetBits(GPIOB,GPIO_Pin_5); //LED off
}
```

Do it by yourself! ! !

写函数的过程：需要背过函数名吗？需要一个字母一个字母的敲入吗？

把GPIO端口设置这段代码删掉试一下自己写。

正常使用主观题需2.0以上版本雨课堂

作答



Part II : Further Improving of LED Program based on FWLib

main.c

```
#include "stm32f10x.h"
int main(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    // 打开 GPIOB 端口的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB,ENABLE);
    // 配置IO口为输出
    GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    // 控制 ODR 寄存器
    GPIO_ResetBits(GPIOB,GPIO_Pin_5); //LED on
    //GPIO_SetBits(GPIOB,GPIO_Pin_5); //LED off
}
```

How long does it take you to understand the function of this function ? ?

How long does it take you to understand the function of this function now ? ?

How long does it take you if you want to transplant this program ? ?

```
main.c
1  /******
2  芯片: STM32F103VET6
3  实现功能: 低电平点亮LED灯
4  引脚: PB5
5  *****/
6
7  #include "stm32f10x.h"
8  #include "led.h"
9
10 int main(void)
11 {
12     LED_GPIO_Config();
13     LED_ON();
14     //LED_OFF();
15 }
16
```

```
led.h
1  #ifndef __LED_H
2  #define __LED_H
3
4  #include "stm32f10x.h"
5
6  #define LED_GPIO_CLK    RCC_APB2Periph_GPIOB
7  #define LED_GPIO_Pin    GPIO_Pin_5
8  #define LED_GPIO_PORT    GPIOB
9
10 void LED_GPIO_Config(void);
11 void LED_ON(void);
12 void LED_OFF(void);
13 #endif
```

How long does it take you to understand the function of this function? ?

How long does it take you if you want to transplant this program? ?

```
led.c
1  #include "led.h"
2
3  void LED_GPIO_Config(void)
4  {
5      GPIO_InitTypeDef GPIO_InitStructure;
6      // 打开 GPIOB 端口的时钟
7      RCC_APB2PeriphClockCmd(LED_GPIO_CLK, ENABLE);
8      // 配置IO口为输出
9      GPIO_InitStructure.GPIO_Mode=GPIO_Mode_Out_PP;
10     GPIO_InitStructure.GPIO_Pin=LED_GPIO_Pin;
11     GPIO_InitStructure.GPIO_Speed=GPIO_Speed_10MHz;
12     GPIO_Init(LED_GPIO_PORT, &GPIO_InitStructure);
13 }
14
15
16 void LED_ON(void)
17 {
18     // 输出电平
19     GPIO_ResetBits(LED_GPIO_PORT, LED_GPIO_Pin); //LED on
20 }
21
22
23
24 void LED_OFF(void)
25 {
26     // 输出电平
27     GPIO_SetBits(LED_GPIO_PORT, LED_GPIO_Pin); //LED off
28 }
29
```

Summary ! !

Modularization steps (LED program as an example)

- ~~1. Create a new folder LED (led.c and led.h) under the USER folder~~
2. Add led.c to the project, and write the include statement---
`#include "led.h"`
3. Write corresponding header files and source files(led.h led.c)
4. Finally, complete the main function. Note that the header file of the module should be added to the main function: `#include "led.h"`
- ~~5. Add the header file path of led.h~~



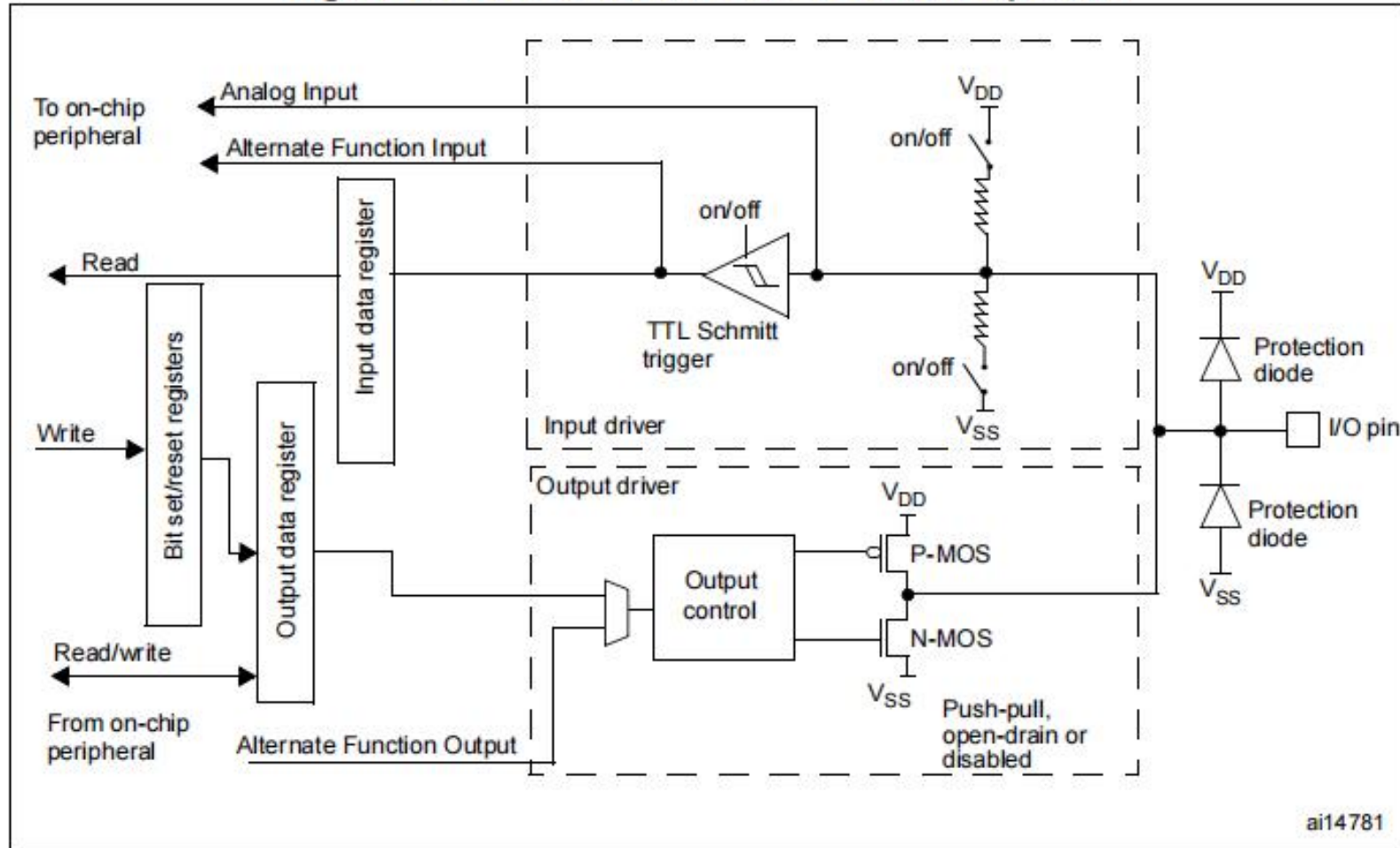
Finally, the LED Lab is completed now ! ! !

Good Programming Habit

- **Before programming, order your thought and draw the flow chart;**
- **Write comments;**
- **Modularization ;**
- **Consider the better transplantation of the code ;**

Part III : Key Detection ---> Control LED on/off

Figure 13. Basic structure of a standard I/O port bit



要配置一个端口为上拉输入模式，需要配置那几个寄存器？



Table 20. Port bit configuration table

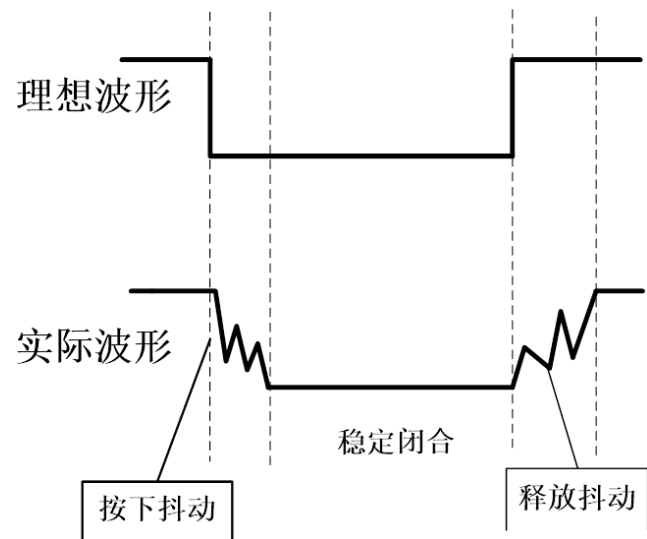
Configuration mode		CNF1	CNF0	MODE1	MODE0	PxODR register	
General purpose output	Push-pull	0	0	01 10 11 see Table 21		0 or 1	
	Open-drain		1			0 or 1	
Alternate Function output	Push-pull	1	0				Don't care
	Open-drain		1				Don't care
Input	Analog	0	0	00		Don't care	
	Input floating		1			Don't care	
	Input pull-down	1	0			0	
	Input pull-up					1	

Table 21. Output MODE bits

MODE[1:0]	Meaning
-----------	---------

STM32F103 GPIO 引脚工作模式真值表分析											
			十六进制	二进制							
				bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
					上/下拉		输入/出	具体工作模式 对照寄存器说明			
GPIO_Mode_TypeDef	GPIO_Mode_AIN	模拟输入	0X00	0	0	0	0	0	0	0	0
	GPIO_Mode_IN_FLOATING	浮空输入	0X04	0	0	0	0	0	1	0	0
	GPIO_Mode_IPD	下拉输入	0X28	0	0	1	0	0	0	0	0
	GPIO_Mode_IPU	上拉输入	0X48	0	1	0	0	0	0	0	0
	GPIO_Mode_Out_OD	开漏输出	0X14	0	0	0	1	0	1	0	0
	GPIO_Mode_Out_PP	推挽输出	0X10	0	0	0	1	0	0	0	0
	GPIO_Mode_AF_OD	复用开漏输出	0X1C	0	0	0	1	1	1	0	0
	GPIO_Mode_AF_PP	复用推挽输出	0X18	0	0	0	1	1	0	0	0
这8个宏的高4bit可随意设置，只要能在程序上帮助判断出模式即可，真正写到寄存器的值是bit2和bit3											



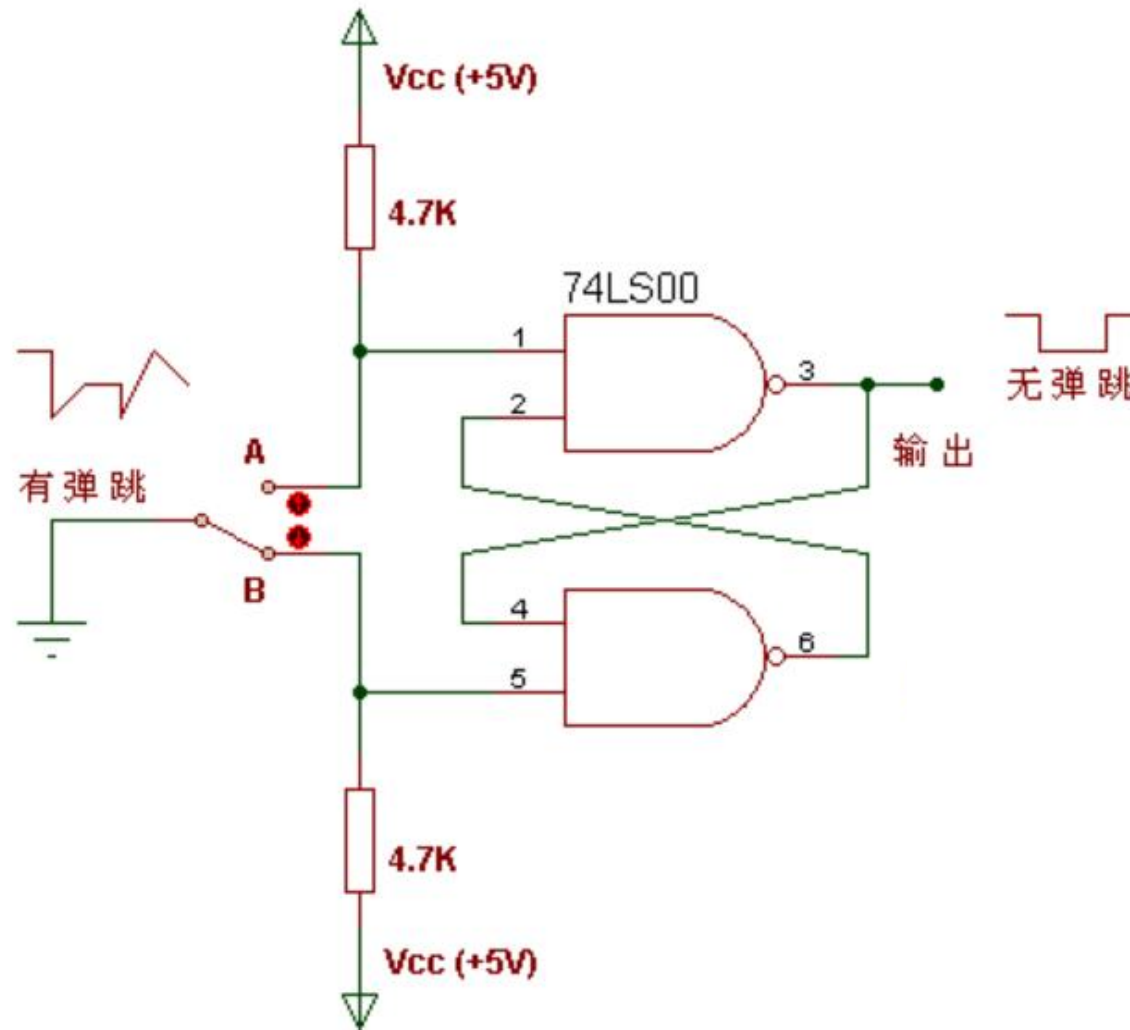


jitter of keys

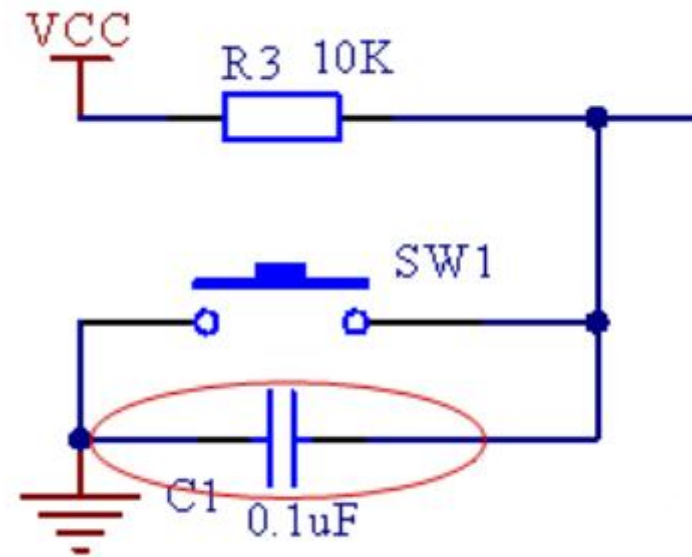
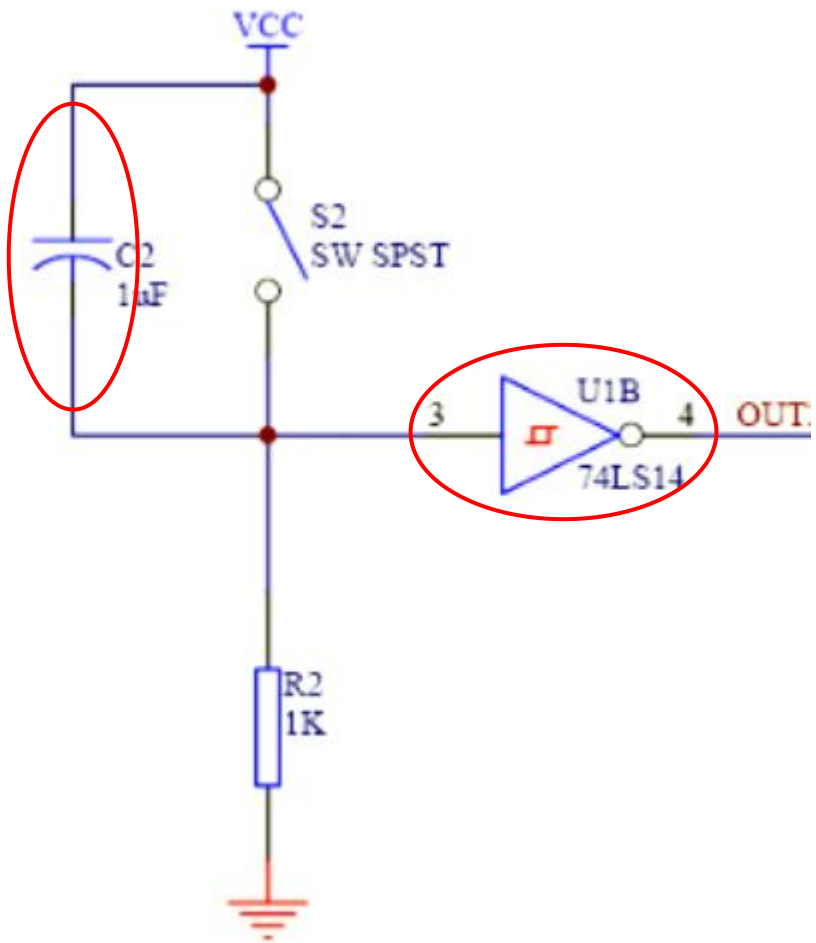
*Eliminate the
Keys Jitter*

Hardware
software

Eliminate the Keys Jitter by Hardware



SPDT Switch



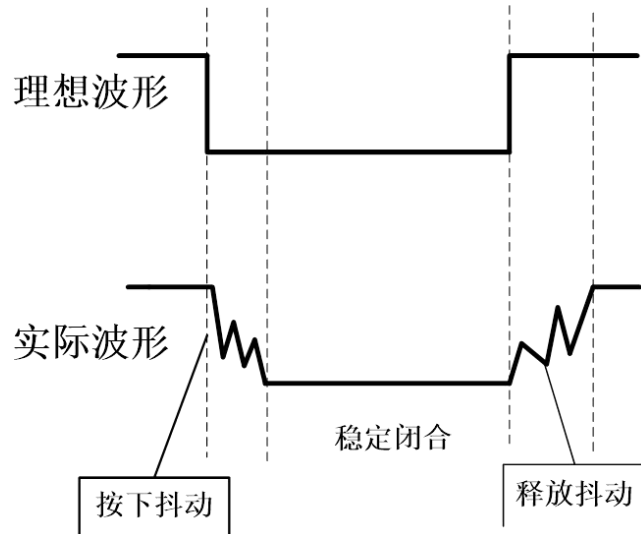
SPST Switch or Keys

Example

```
uint8_t Key_Scan(GPIO_TypeDef *GPIOx,uint16_t GPIO_Pin)
{
    if( GPIO_ReadInputDataBit(GPIOx, GPIO_Pin) == KEY_ON )
    {
        while( GPIO_ReadInputDataBit(GPIOx, GPIO_Pin) == KEY_ON ); // 松手检测

        return KEY_ON;
    }
    else return KEY_OFF;
}
```

Eliminate the Keys Jitter by Software



- ◆ Detect the key , if it is low
- ◆ Delay 5ms
- ◆ Check whether the key is still low
- ◆ If it is low, we can confirm the key is pressed

Example

```
uint8_t Key_Scan(GPIO_TypeDef *GPIOx,uint16_t GPIO_Pin)
{
    if( GPIO_ReadInputDataBit(GPIOx, GPIO_Pin) == KEY_ON )
    {
        Delay_ms(5);
        if(GPIO_ReadInputDataBit(GPIOx, GPIO_Pin) == KEY_ON)
        {
            // 松手检测
            while( GPIO_ReadInputDataBit(GPIOx, GPIO_Pin) == KEY_ON );
            return KEY_ON;
        }
        else
        {
            return KEY_OFF;
        }
    }
    else
    {
        return KEY_OFF;
    }
}
```

Homework of Lecture4

- LED turn on and off alternately (Based on FWLib and Modularization, use a delay function to generate a time delay)
- Press the key to control the change between LED on and off (in the report , the programming flow chart is necessary too)

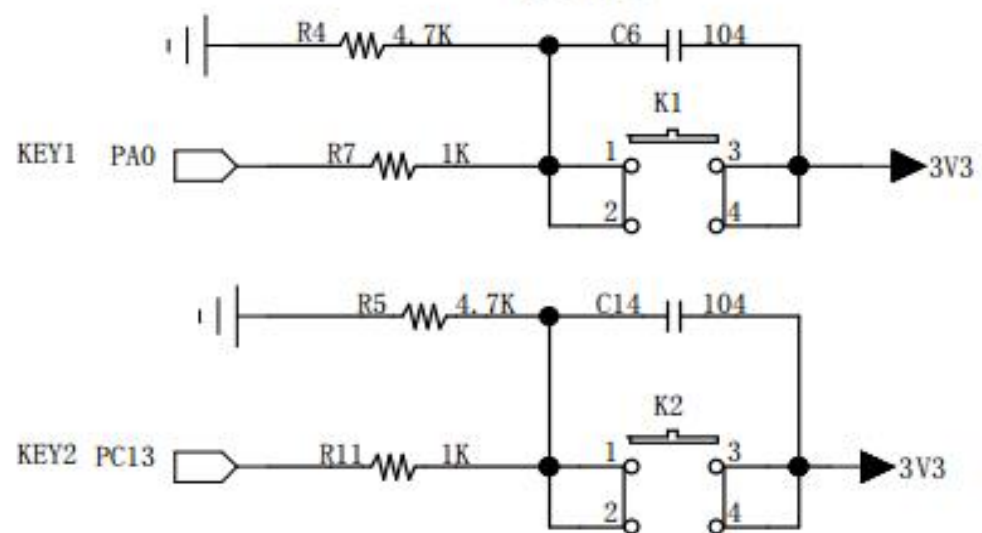
main.c

```
#include "stm32f10x.h"
#include "led.h"

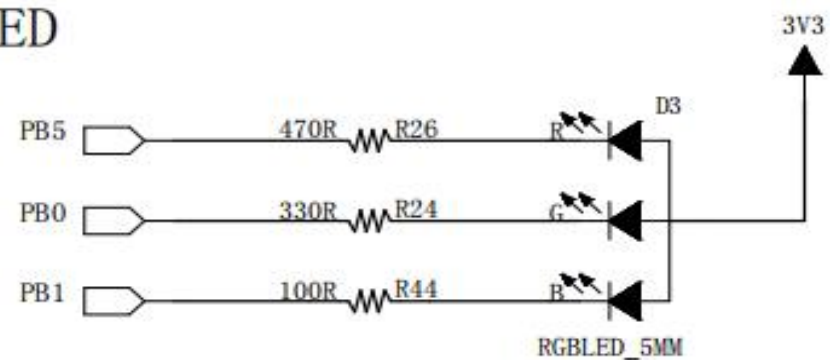
void Delay( uint32_t count )
{
    for(; count!=0; count--);
}

int main(void)
{
    LED_GPIO_Config();
    while(1)
    {
        LED_ON();
        Delay(0xffffffff);
        LED_OFF();
        Delay(0xffffffff);
    }
}
```

按键



LED



TIM3_CH3	LED_G	PB0
TIM3_CH4	LED_B	PB1
TIM3_CH2	LED_R	PB5

Now , You can do many many things.



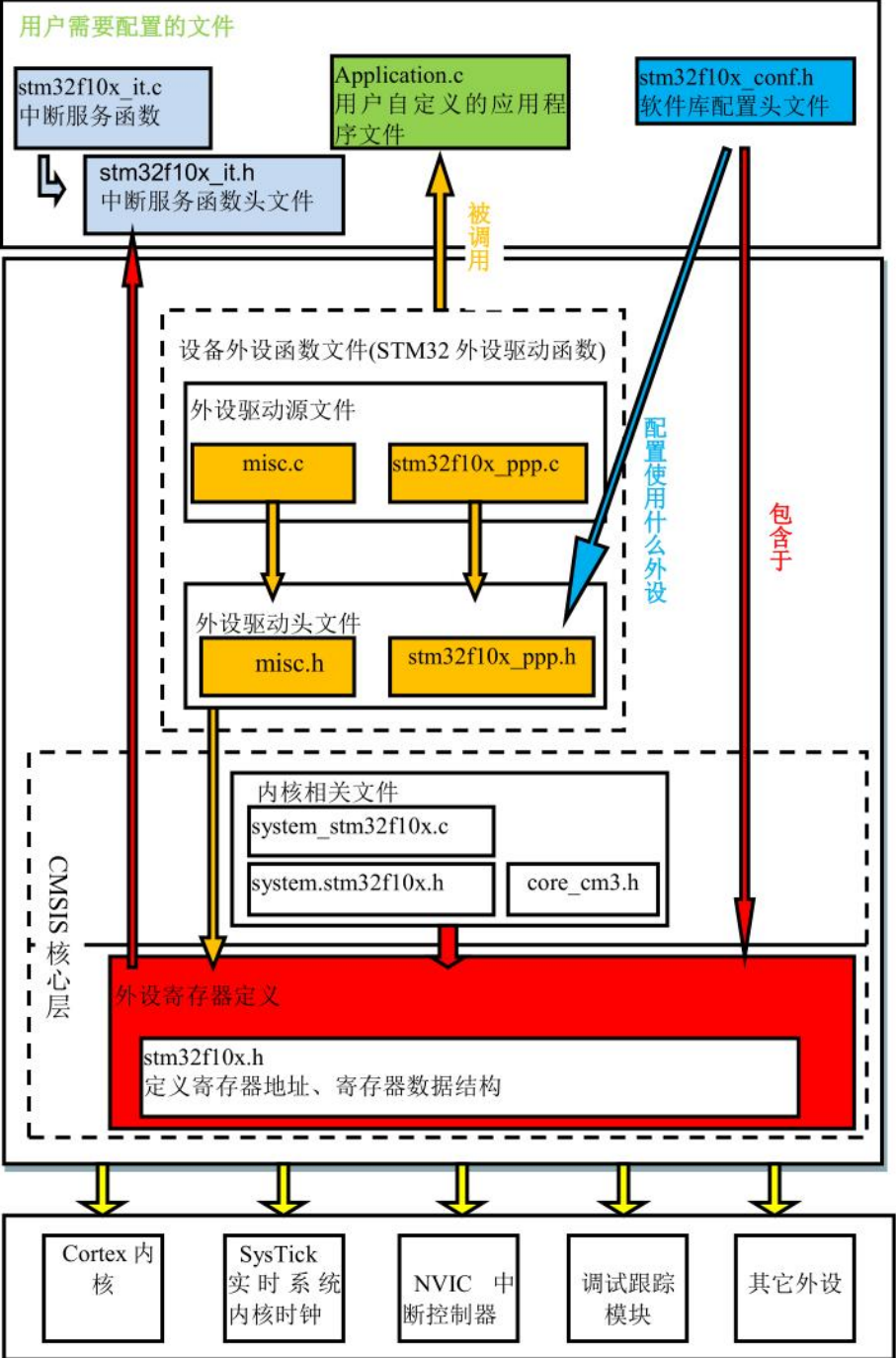
startup file

- The start-up file is necessary , and it is the entrance of the project;
- There is no need to modify, but to select the startup file matched with the chip used;
- Location: ..\官方固件库\STM32F10x_StdPeriph_Lib_V3.6.0 \Libraries \CMSIS\CM3 \DeviceSupport\ST\STM32F10x\startup\arm [Download URL](#)
- Which one : startup_stm32f10x_hd.s

startup_stm32f10x_cl.s
startup_stm32f10x_hd.s
startup_stm32f10x_hd_vl.s
startup_stm32f10x_ld.s
startup_stm32f10x_ld_vl.s
startup_stm32f10x_md.s
startup_stm32f10x_md_vl.s
startup_stm32f10x_xl.s

启动文件	区别
startup_stm32f10x_ld.s	ld: low-density 小容量, FLASH 容量在 16-32K 之间
startup_stm32f10x_md.s	md: medium-density 中容量, FLASH 容量在 64-128K 之间
startup_stm32f10x_hd.s	hd: high-density 大容量, FLASH 容量在 256-512K 之间
startup_stm32f10x_xl.s	xl: 超大容量, FLASH 容量在 512-1024K 之间
以上四种都属于基本型, 包括 STM32F101xx、STM32F102xx、STM32F103xx 系列	
startup_stm32f10x_cl.s	cl:connectivity line devices 互联型, 特指 STM32F105xx 和 STM32F107xx 系列
startup_stm32f10x_ld_vl.s	vl:value line devices 超值型系列, 特指 STM32F100xx 系列
startup_stm32f10x_md_vl.s	
startup_stm32f10x_hd_vl.s	





`stm32f10x_it.c, stm32f10x_it.h`
`stm32f10x_conf.h`
`stm32f10x_ppp.c, stm32f10x_ppp.h`
`misc.c, misc.h`
`core_cm3.c, core_cm3.h`
`system_stm32f10x_it.c, system_stm32f10x_it.h`
`stm32f10x.h`
`startup_stm32f10x_hd.s`

STM32F10x固件库文件分析		
启动文件	startup_stm32f10x_hd.s	汇编编写的启动文件
外设相关	stm32f10x.h	外设寄存器定义
	system_stm32f10x.h	1、用于系统初始化
	system_stm32f10x.c	2、配置系统时钟
	stm32f10x_xxx.h	外设固件库头文件, 如GPIO, ADC, I2C, SPI等
	stm32f10x_xxx.c	外设的固件库文件, 如GPIO, ADC, I2C, SPI等
内核相关	core_cm3.h	内核寄存器定义
	core_cm3.c	控制内核外设的相关函数, 用的非常少
	misc.h	跟NVIC和SysTick相关的函数
	misc.c	
用户相关	main.c	main函数存在的地方
	stm32f10x_it.h	用户编写的中断服务函数都放在这里
	stm32f10x_it.c	

1-汇编编写的启动文件

startup_stm32f10x_hd.s: 设置堆栈指针、设置PC指针、初始化中断向量表、配置系统时钟、调用C库函数_main最终去到C的世界

2-时钟配置文件

system_stm32f10x.c: 把外部时钟HSE=8M, 经过PLL倍频为72M。

3-外设相关的

stm32f10x.h: 实现了内核之外的外设的寄存器映射

xxx: GPIO、USART、I2C、SPI、FSMC

stm32f10x_xx.c: 外设的驱动函数库文件

stm32f10x_xx.h: 存放外设的初始化结构体, 外设初始化结构体成员的参数列表, 外设固件库函数的声明

4-内核相关的

CMSIS - Cortex 微控制器软件接口标准

core_cm3.h: 实现了内核里面外设的寄存器映射

core_cm3.c

NVIC(嵌套向量中断控制器)、SysTick(系统滴答定时器)

misc.h

misc.c

5-头文件的配置文件

stm32f10x_conf.h: 头文件的头文件

//stm32f10x_usart.h

//stm32f10x_i2c.h

//stm32f10x_spi.h

//stm32f10x_adc.h

//stm32f10x_fsmc.h

.....

6-专门存放中断服务函数的c文件

stm32f10x_it.c

stm32f10x_it.h

中断服务函数你可以随意放在其他的地方, 并不是一定要放在stm32f10x_it.c